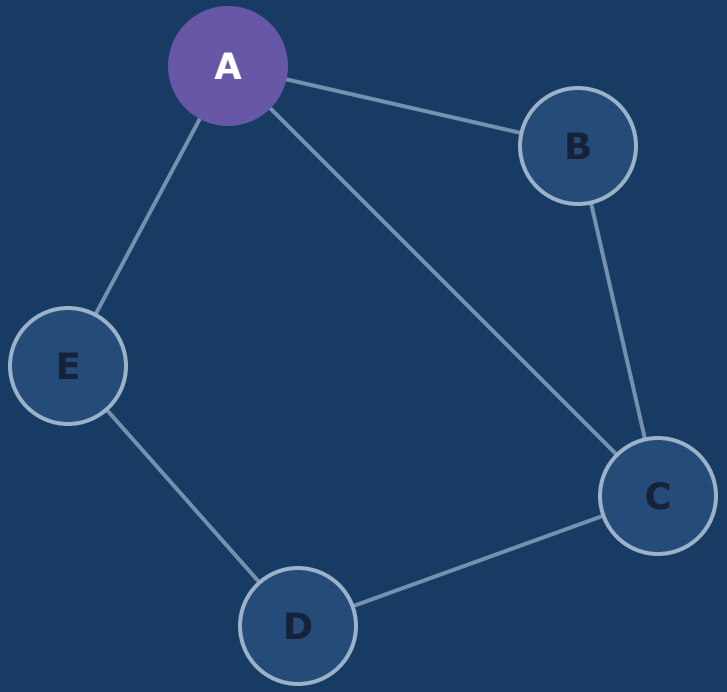
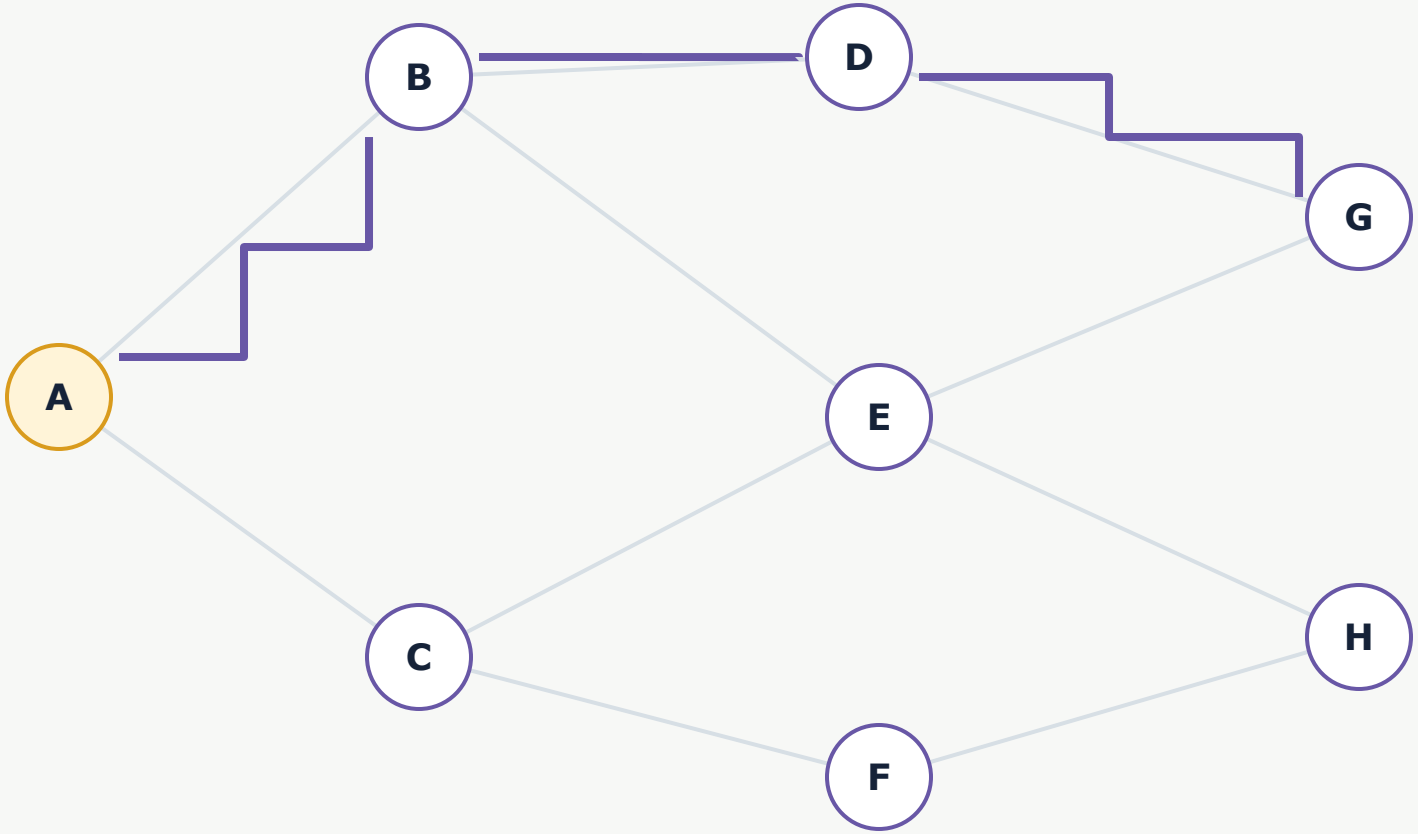
Busca em profundidade

Percurso com retrocesso

Como recursão ou pilha explora um caminho até o fim e retorna para continuar o percurso.



Exploração de um ramo

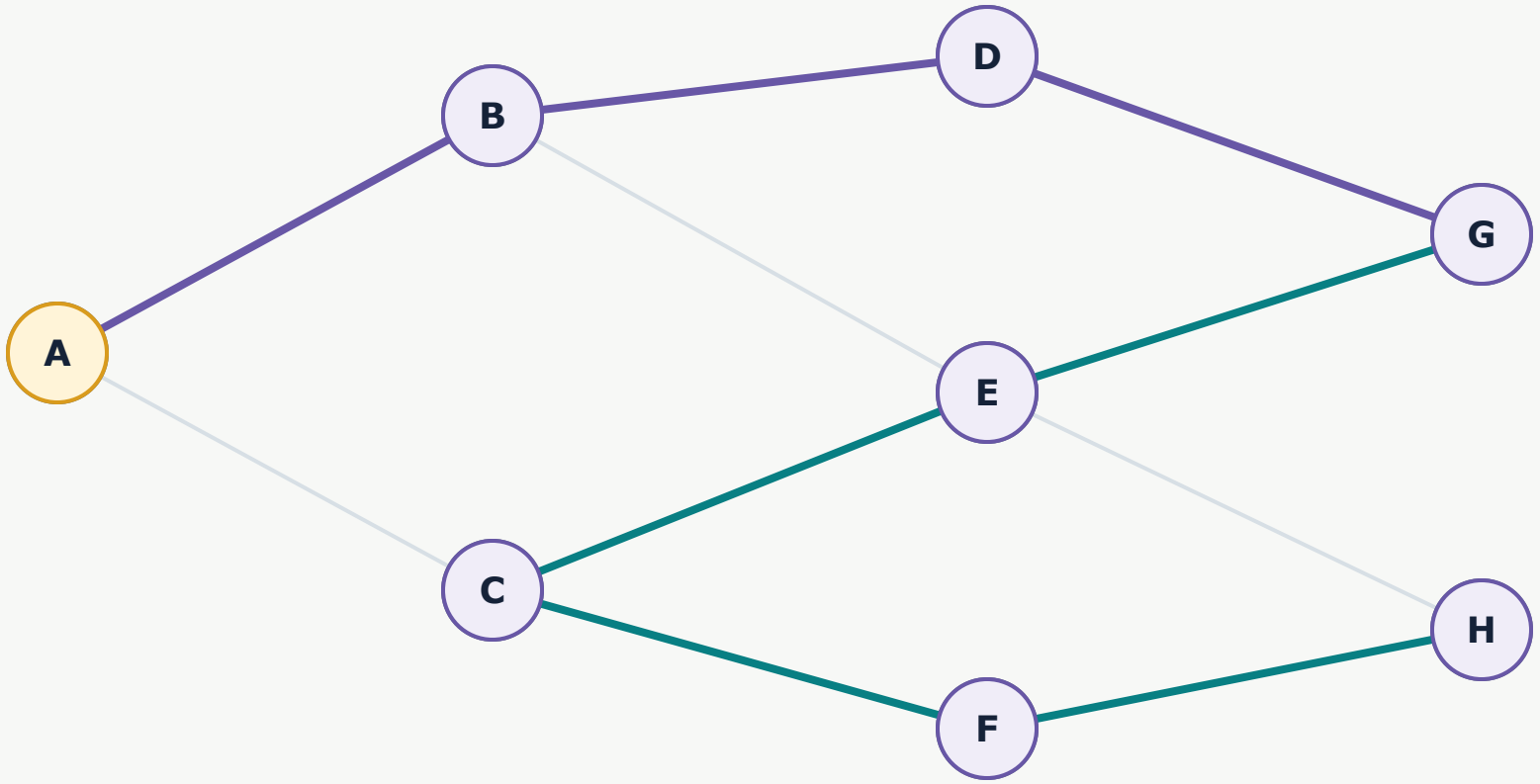


Ideia

A busca escolhe um vizinho ainda não visitado e continua a partir dele. Quando não há opção, retorna ao antecessor.

No exemplo, o início pode seguir A, B, D, G antes do primeiro retorno.

Profundidade antes de largura

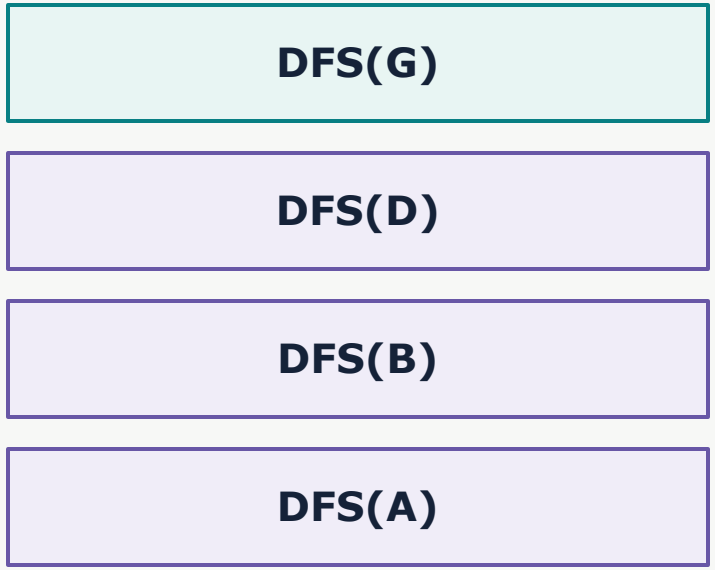


Uma ordem possível: A, B, D, G, E, C, F, H

Recursão e pilha

Cada chamada ativa guarda o vértice atual e o ponto em que sua lista de vizinhos foi interrompida.

topo da pilha



Descida

Enquanto DFS(G) está ativa, ela chama DFS(E). Cada retorno retoma a lista de vizinhos do vértice anterior.

Versão iterativa

Uma pilha explícita reproduz a mesma disciplina LIFO. A ordem exata exige atenção ao momento em que os vizinhos são inseridos.

Estados dos vértices

Três estados tornam explícito o ciclo de vida de cada vértice.

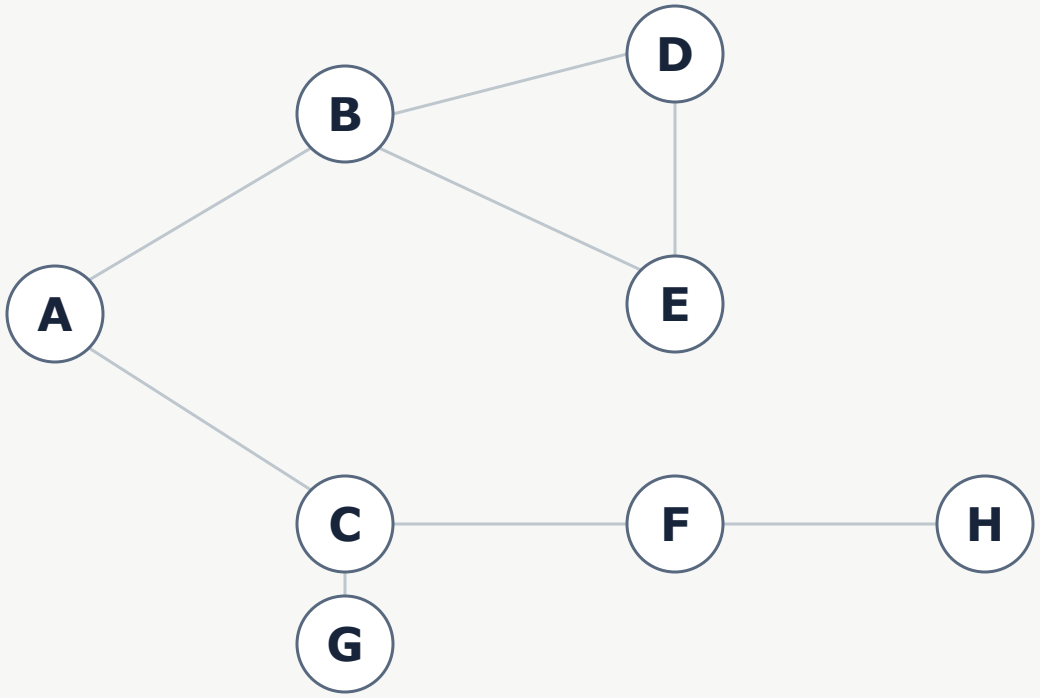


Uso dos estados

Uma aresta para vértice em processamento revela retorno a um ancestral em grafos dirigidos. Isso caracteriza um ciclo.

Grafo e regras do percurso

Grafo não dirigido, início em A



Vizinhos em ordem alfabética

A: B, C
B: A, D, E
C: A, F, G
D: B, E
E: B, D
F: C, H
G: C
H: F

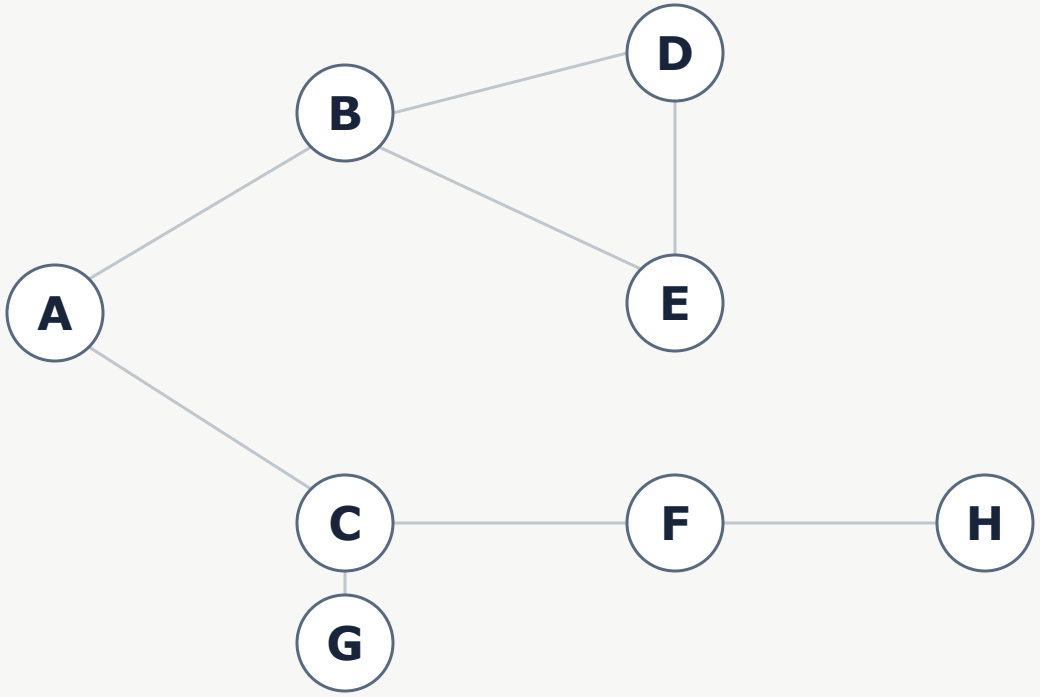
Uma chamada só termina depois de examinar todos os seus vizinhos.

O exemplo inclui o ciclo B–D–E–B e dois ramos em A. O retorno a A permite continuar por C.

Cada slide seguinte mostra o estado imediatamente após a ação indicada no título.

00 Antes da primeira chamada

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

vazia

Inicialização

Todos os vértices estão não descobertos. A pilha está vazia.

A primeira chamada será visitar(A).

d: tempo de descoberta.
f: tempo de término.
O relógio avança a cada descoberta ou finalização.

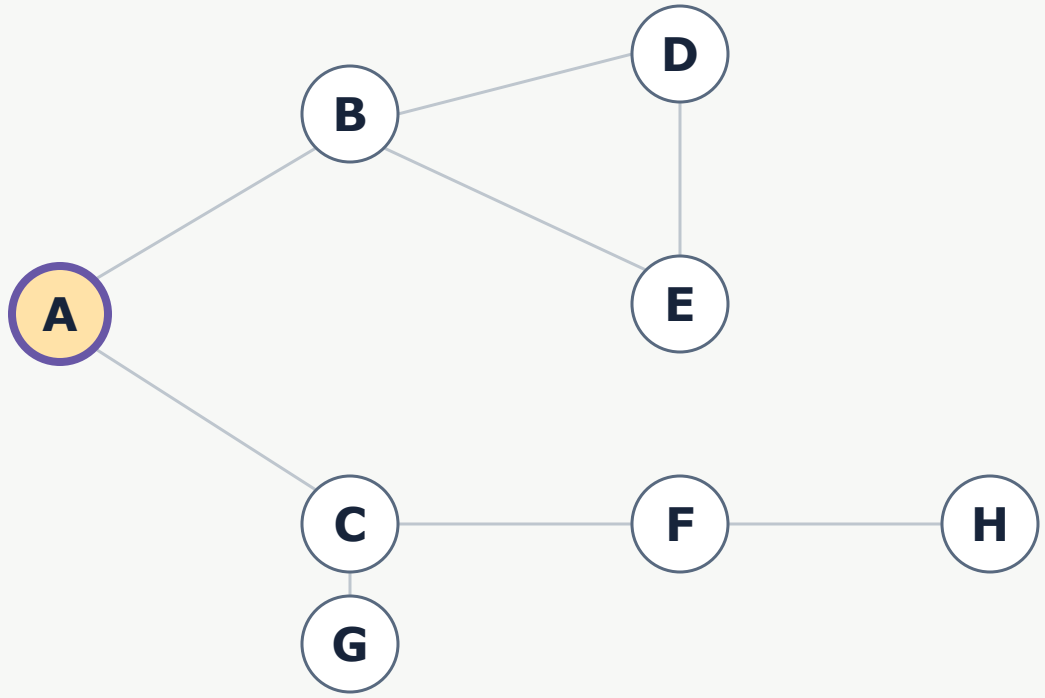
Vértice	A	B	C	D	E	F	G	H
Estado	N	N	N	N	N	N	N	N
pai	—	—	—	—	—	—	—	—
d / f	— / —	— / —	— / —	— / —	— / —	— / —	— / —	— / —

Descoberta: —

Término: —

01 Descoberta de A

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(A) ← topo base

Relógio: t = 1

A inicia a busca. Seu primeiro vizinho é B.

C permanece não descoberto enquanto o ramo de B estiver ativo.

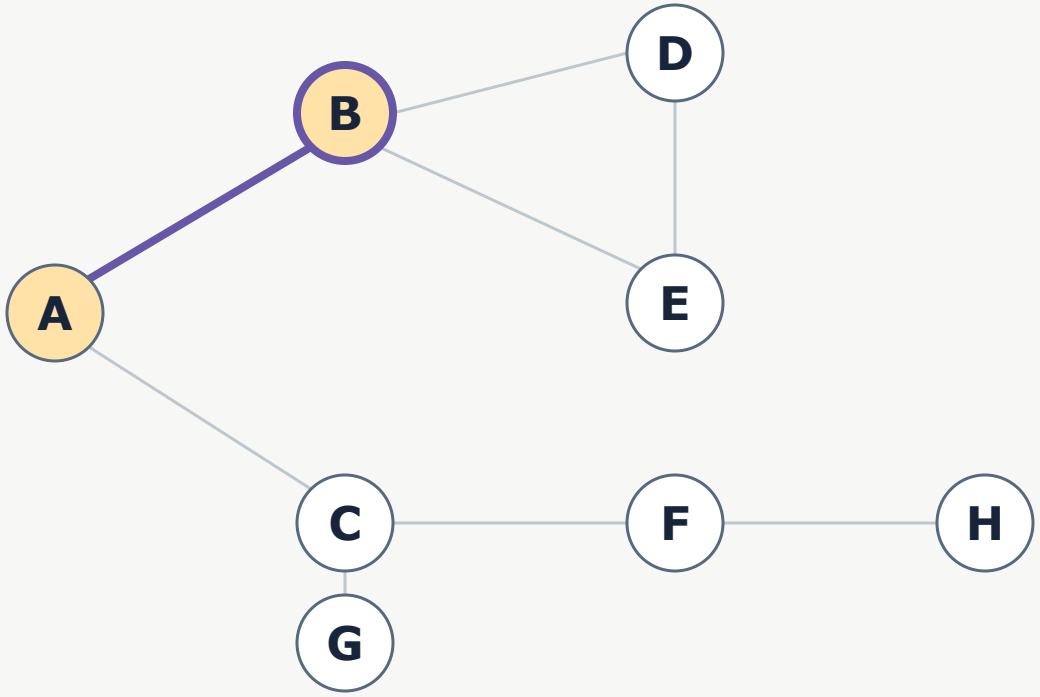
Vértice	A	B	C	D	E	F	G	H
Estado	E	N	N	N	N	N	N	N
pai	—	—	—	—	—	—	—	—
d / f	1 / —	— / —	— / —	— / —	— / —	— / —	— / —	— / —

Descoberta: A

Término: —

02 Descoberta de B

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(B) ← topo

visitar(A) base

Relógio: t = 2

A chama visitar(B). Registre pai[B] = A.

B ignora A, seu pai, e escolhe D. A aguarda o retorno para examinar C.

nova chamada

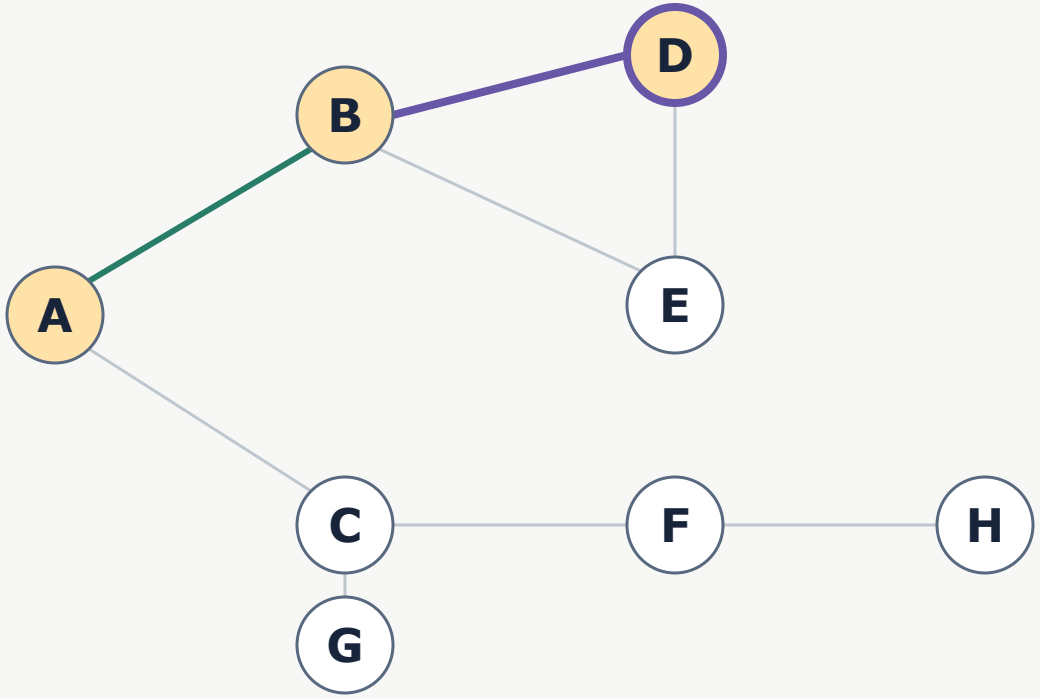
Vértice	A	B	C	D	E	F	G	H
Estado	E	E	N	N	N	N	N	N
pai	—	A	—	—	—	—	—	—
d / f	1 / —	2 / —	— / —	— / —	— / —	— / —	— / —	— / —

Descoberta: A, B

Término: —

03 Descoberta de D

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(D) ← topo

visitar(B)

visitar(A) base

nova chamada

Relógio: t = 3

B chama visitar(D). Registre pai[D] = B.

D ignora B, seu pai, e escolhe E. B aguarda para examinar E.

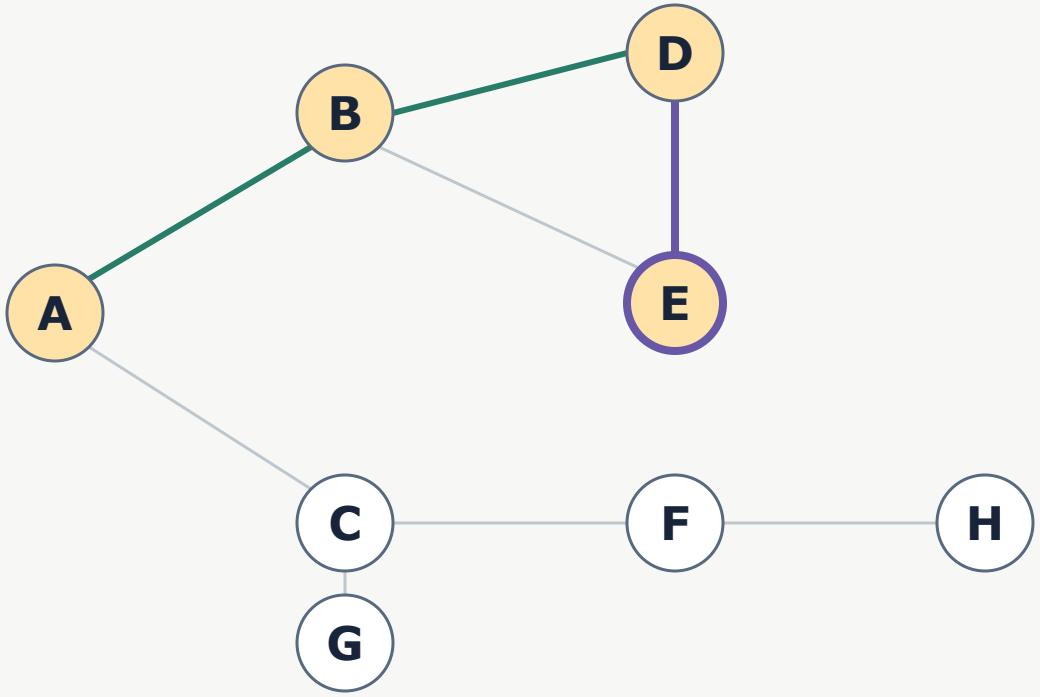
Vértice	A	B	C	D	E	F	G	H
Estado	E	E	N	E	N	N	N	N
pai	—	A	—	B	—	—	—	—
d / f	1 / —	2 / —	— / —	3 / —	— / —	— / —	— / —	— / —

Descoberta: A, B, D

Término: —

04 Descoberta de E

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(E) ← topo

visitar(D)

visitar(B)

visitar(A) base

Relógio: t = 4

D chama visitar(E). Registre pai[E] = D.

A pilha atinge quatro chamadas. E examina primeiro B, que já está em processamento.

nova chamada

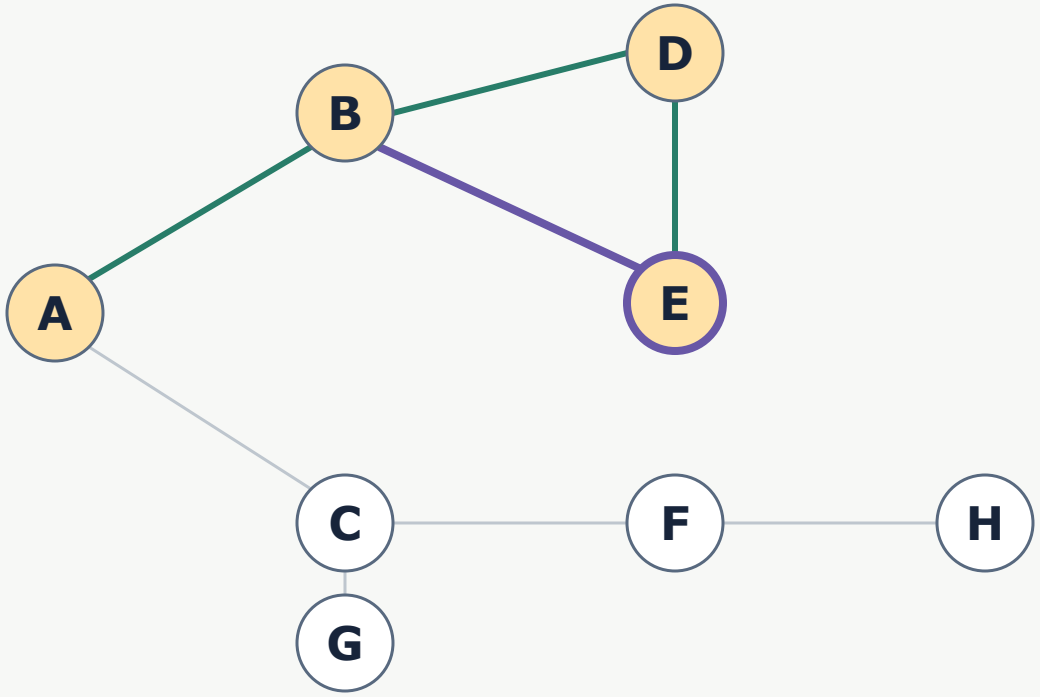
Vértice	A	B	C	D	E	F	G	H
Estado	E	E	N	E	E	N	N	N
pai	—	A	—	B	D	—	—	—
d / f	1 / —	2 / —	— / —	3 / —	4 / —	— / —	— / —	— / —

Descoberta: A, B, D, E

Término: —

05 Uma aresta fecha um ciclo

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



aresta examinada

Pilha de chamadas

Topo: chamada que executa agora

visitar(E) ← topo

visitar(D)

visitar(B)

visitar(A) base

Inspeção sem descoberta

E examina B, que está em processamento e não é seu pai.

A aresta E–B fecha o ciclo B–D–E–B. E ignora essa aresta e continua sua lista.

Não há chamada nova nem avanço do relógio.

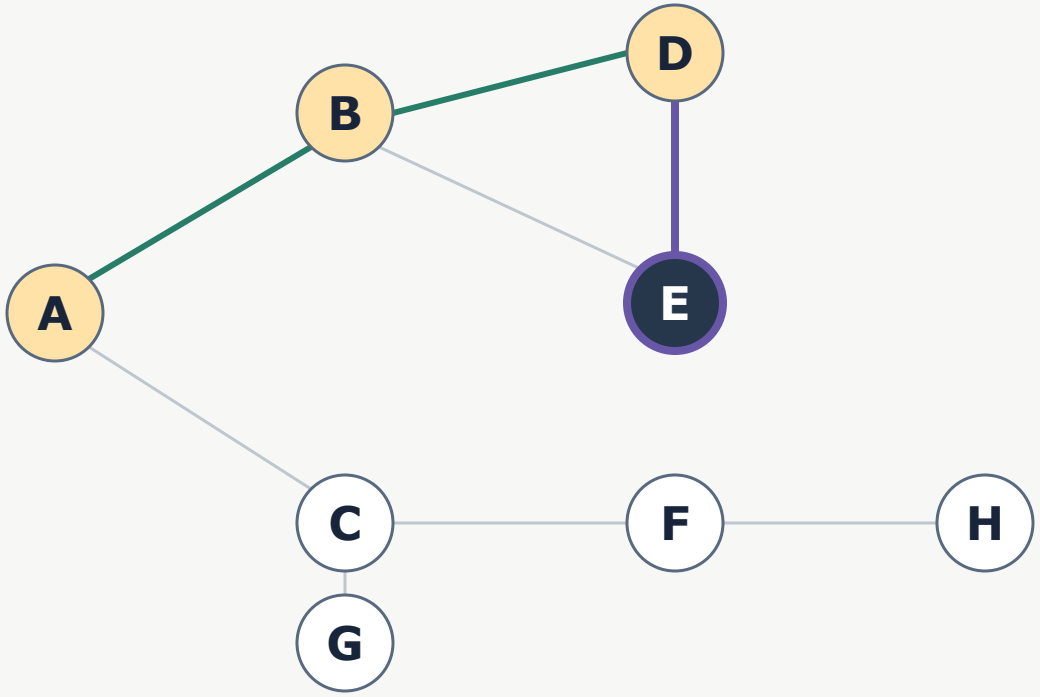
Vértice	A	B	C	D	E	F	G	H
Estado	E	E	N	E	E	N	N	N
pai	—	A	—	B	D	—	—	—
d / f	1 / —	2 / —	— / —	3 / —	4 / —	— / —	— / —	— / —

Descoberta: A, B, D, E

Término: —

06 Finalização de E

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(D) ← topo

visitar(B)

visitar(A) base

Relógio: t = 5

E ignora D, seu pai. Não há vizinho não descoberto.

E passa a finalizado e sai da pilha. visitar(D) retoma a execução.

retorno da chamada

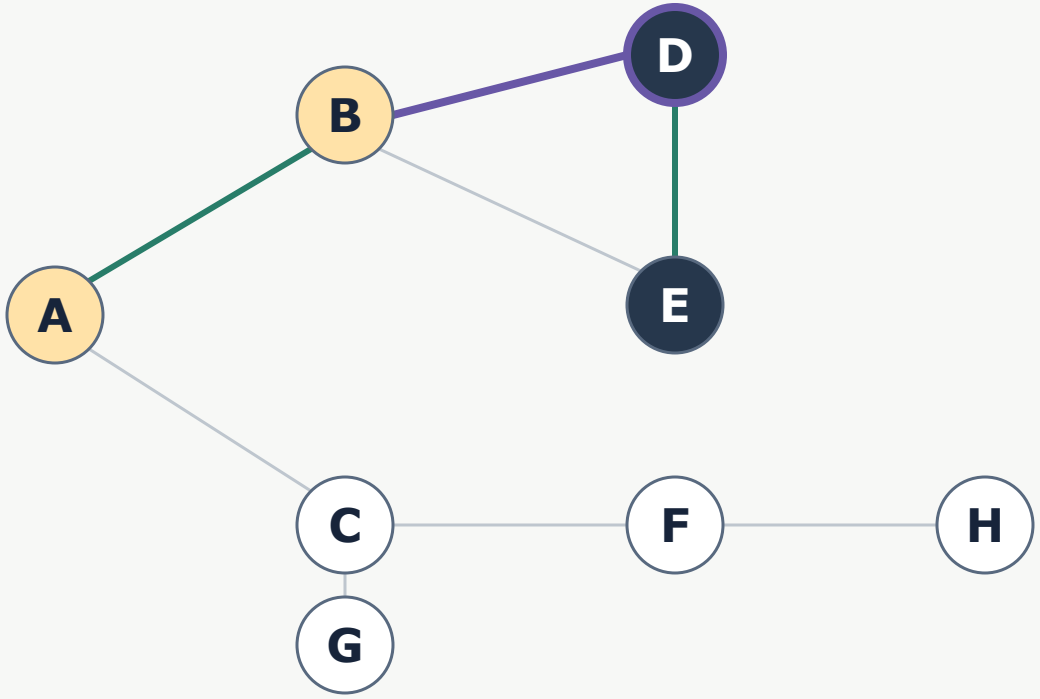
Vértice	A	B	C	D	E	F	G	H
Estado	E	E	N	E	F	N	N	N
pai	—	A	—	B	D	—	—	—
d / f	1 / —	2 / —	— / —	3 / —	4 / 5	— / —	— / —	— / —

Descoberta: A, B, D, E

Término: E

07 Finalização de D

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(B) ← topo

visitar(A) base

Relógio: t = 6

A chamada de E terminou e a lista de D acabou.

D passa a finalizado e sai da pilha. B retoma a lista no vizinho E.

retorno da chamada

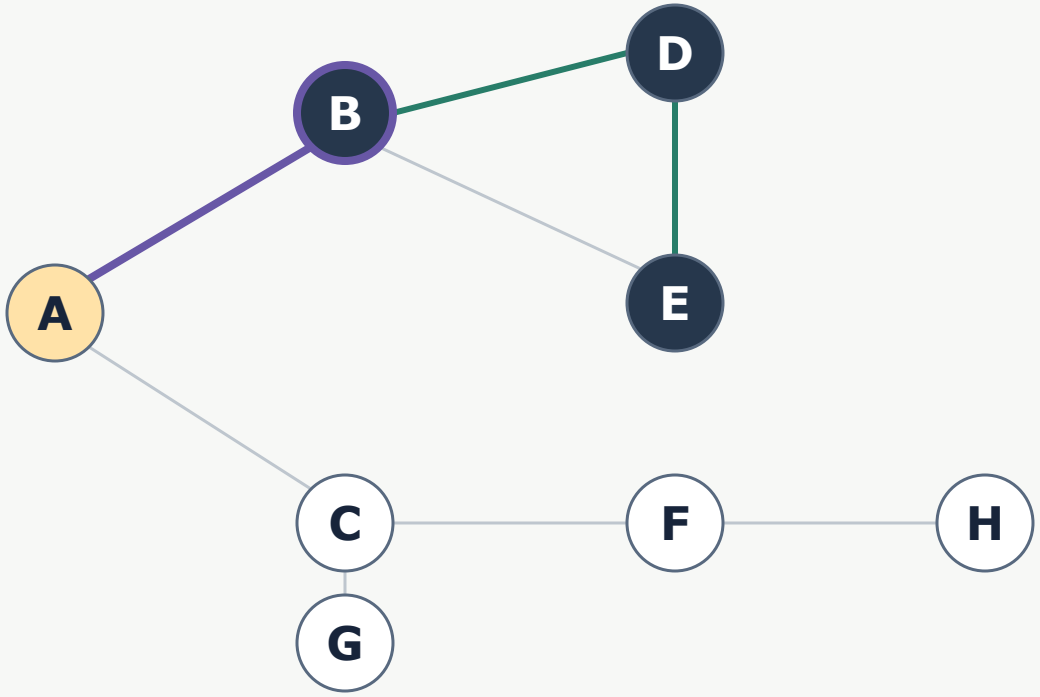
Vértice	A	B	C	D	E	F	G	H
Estado	E	E	N	F	F	N	N	N
pai	—	A	—	B	D	—	—	—
d / f	1 / —	2 / —	— / —	3 / 6	4 / 5	— / —	— / —	— / —

Descoberta: A, B, D, E

Término: E, D

08 Finalização de B

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(A) ← **topo** base

Relógio: t = 7

B examina E, que já está finalizado. Nenhuma nova chamada ocorre.

A lista acabou: B sai da pilha. A retoma a lista no vizinho C.

retorno da chamada

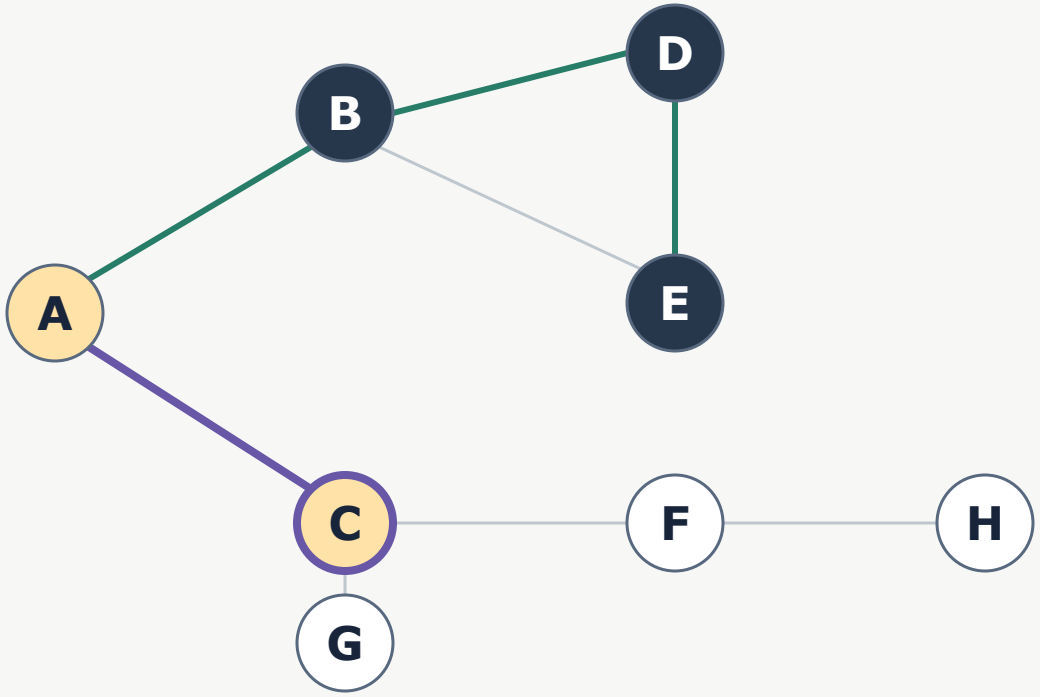
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	N	F	F	N	N	N
pai	—	A	—	B	D	—	—	—
d / f	1 / —	2 / 7	— / —	3 / 6	4 / 5	— / —	— / —	— / —

Descoberta: A, B, D, E

Término: E, D, B

09 Descoberta de C

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(C) ← topo

visitar(A) base

Relógio: t = 8

O ramo de B terminou. A retoma sua lista e chama visitar(C).

C ignora A, seu pai, e escolhe F. O retorno permitiu iniciar um novo ramo.

nova chamada

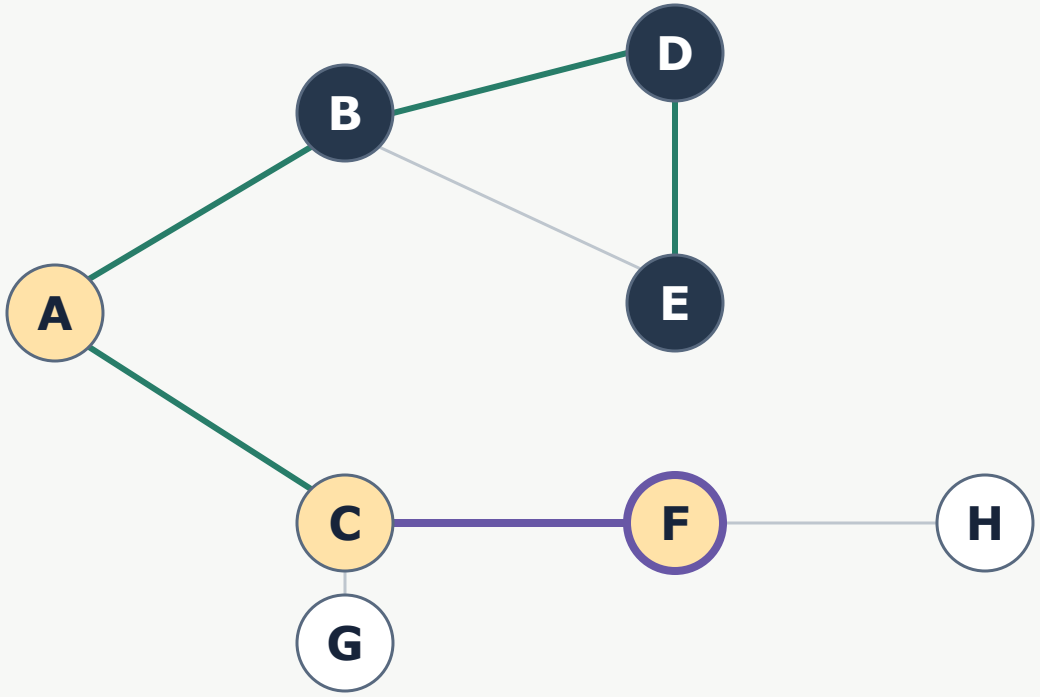
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	E	F	F	N	N	N
pai	—	A	A	B	D	—	—	—
d / f	1 / —	2 / 7	8 / —	3 / 6	4 / 5	— / —	— / —	— / —

Descoberta: A, B, D, E, C

Término: E, D, B

10 Descoberta de F

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(F) ← topo

visitar(C)

visitar(A) base

nova chamada

Relógio: t = 9

C chama visitar(F). Registre pai[F] = C.

F ignora C, seu pai, e escolhe H. C guarda G para depois.

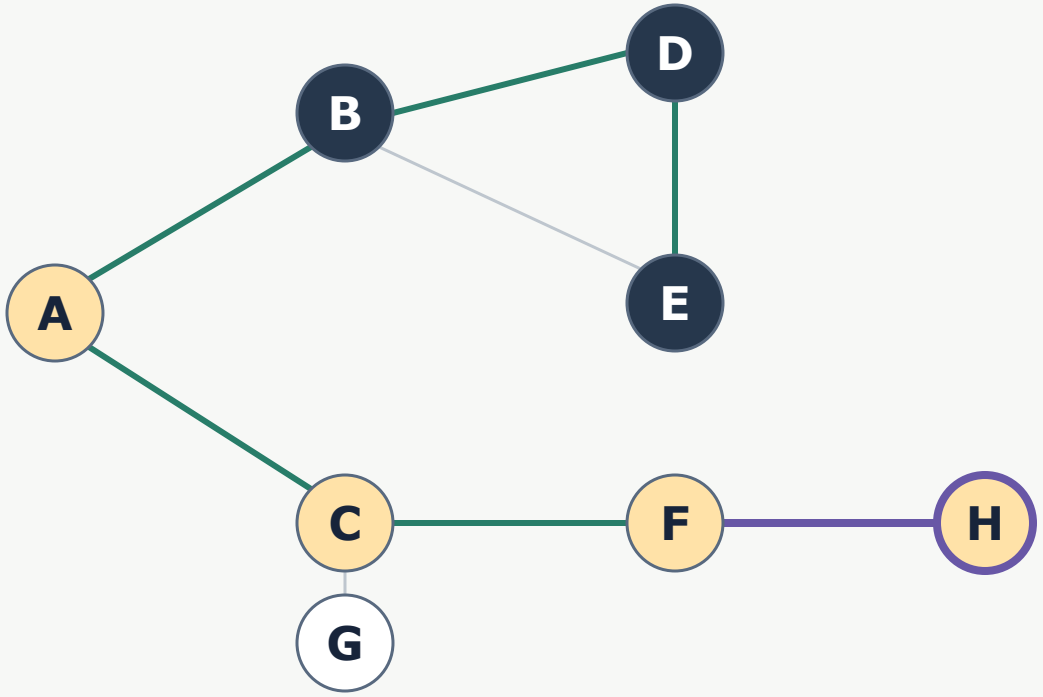
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	E	F	F	E	N	N
pai	—	A	A	B	D	C	—	—
d / f	1 / —	2 / 7	8 / —	3 / 6	4 / 5	9 / —	— / —	— / —

Descoberta: A, B, D, E, C, F

Término: E, D, B

11 Descoberta de H

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(H) ← topo

visitar(F)

visitar(C)

visitar(A) base

Relógio: t = 10

F chama visitar(H). Registre pai[H] = F.

H só possui F como vizinho. Como F é seu pai, nenhuma nova chamada ocorre.

nova chamada

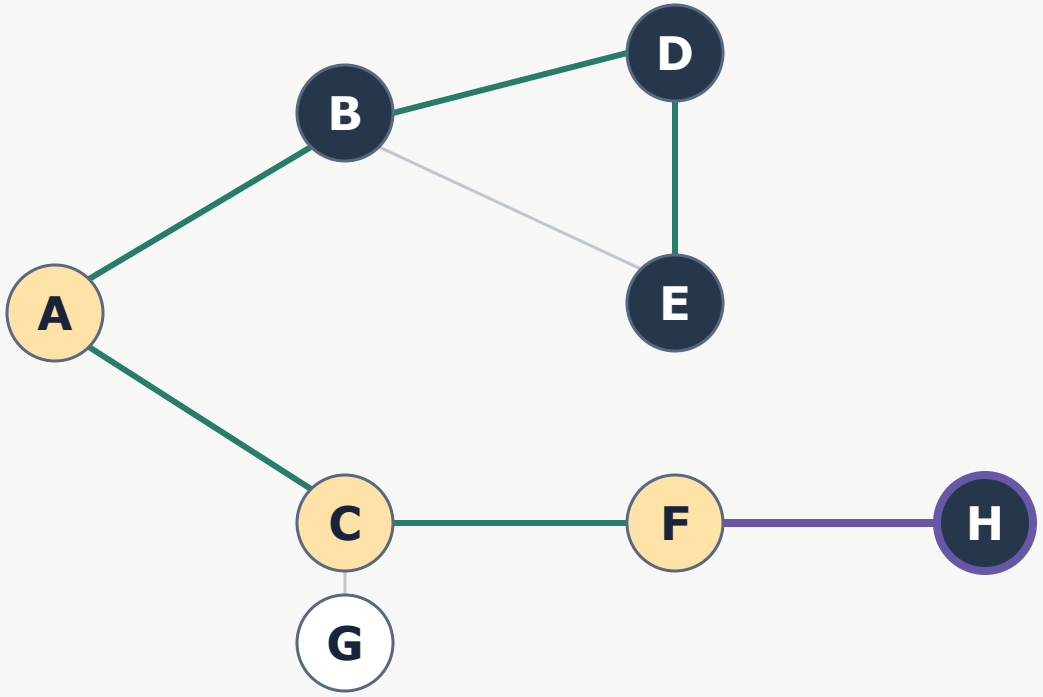
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	E	F	F	E	N	E
pai	—	A	A	B	D	C	—	F
d / f	1 / —	2 / 7	8 / —	3 / 6	4 / 5	9 / —	— / —	10 / —

Descoberta: A, B, D, E, C, F, H

Término: E, D, B

12 Finalização de H

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(F) ← topo

visitar(C)

visitar(A) base

Relógio: t = 11

H terminou de examinar sua lista.

H passa a finalizado e sai da pilha. A execução retorna a F.

retorno da chamada

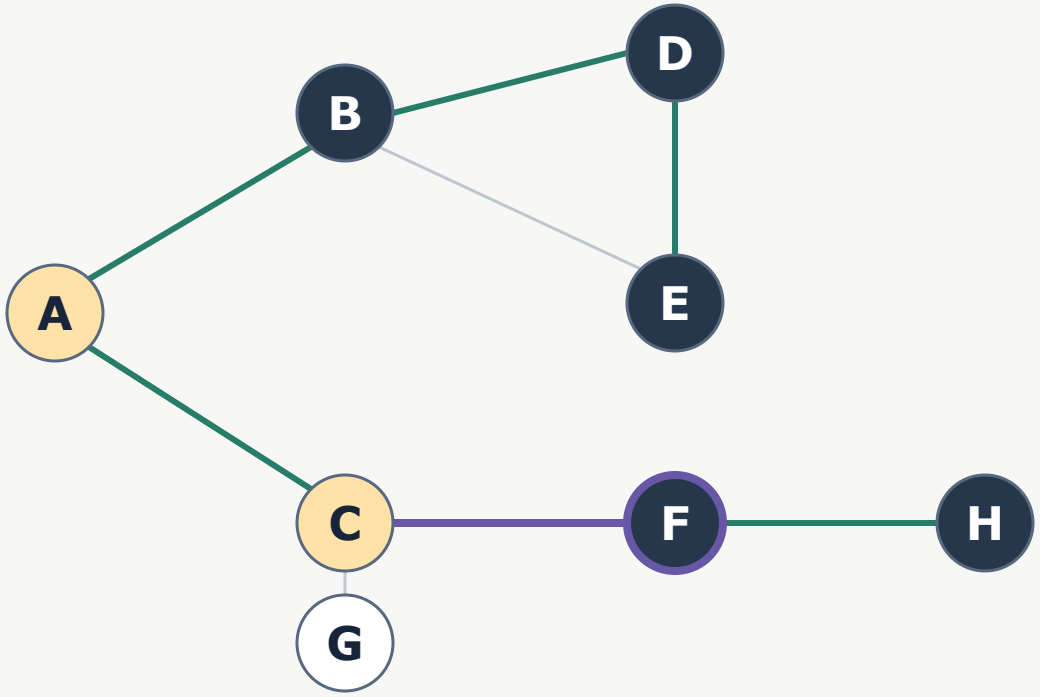
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	E	F	F	E	N	F
pai	—	A	A	B	D	C	—	F
d / f	1 / —	2 / 7	8 / —	3 / 6	4 / 5	9 / —	— / —	10 / 11

Descoberta: A, B, D, E, C, F, H

Término: E, D, B, H

13 Finalização de F

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(C) ← topo

visitar(A) base

Relógio: t = 12

A chamada de H terminou e F não tem outro vizinho.

F passa a finalizado e sai da pilha. C retoma a lista em G.

retorno da chamada

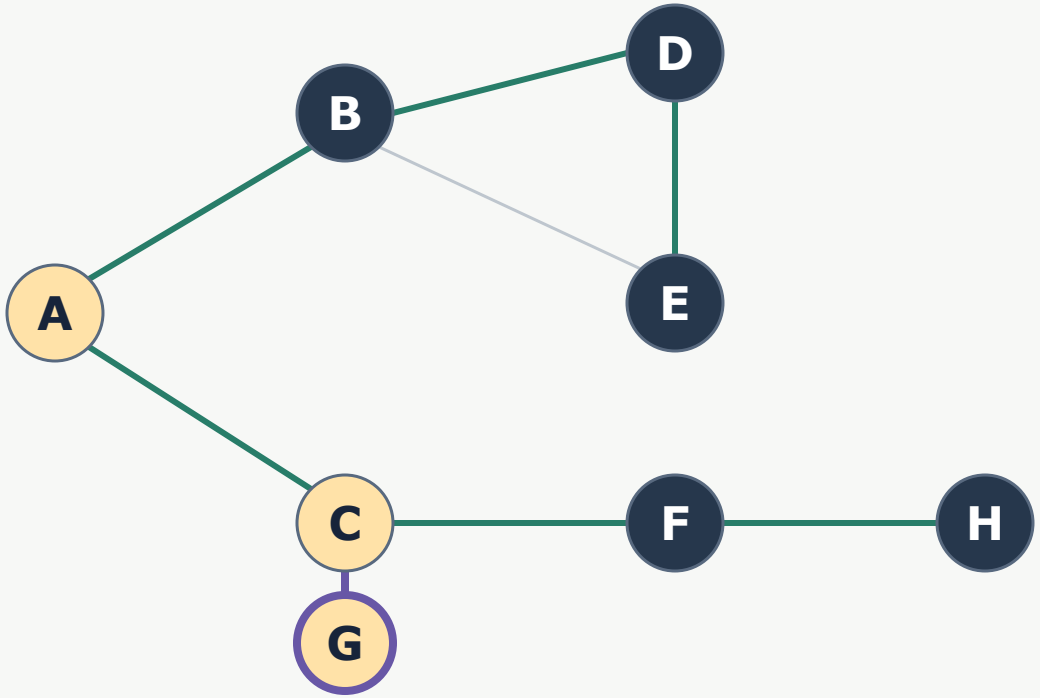
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	E	F	F	F	N	F
pai	—	A	A	B	D	C	—	F
d / f	1 / —	2 / 7	8 / —	3 / 6	4 / 5	9 / 12	— / —	10 / 11

Descoberta: A, B, D, E, C, F, H

Término: E, D, B, H, F

14 Descoberta de G

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(G) ← topo

visitar(C)

visitar(A) base

nova chamada

Relógio: t = 13

C retoma a lista depois de F e chama visitar(G).

G só possui C, seu pai. Ele será finalizado na próxima transição.

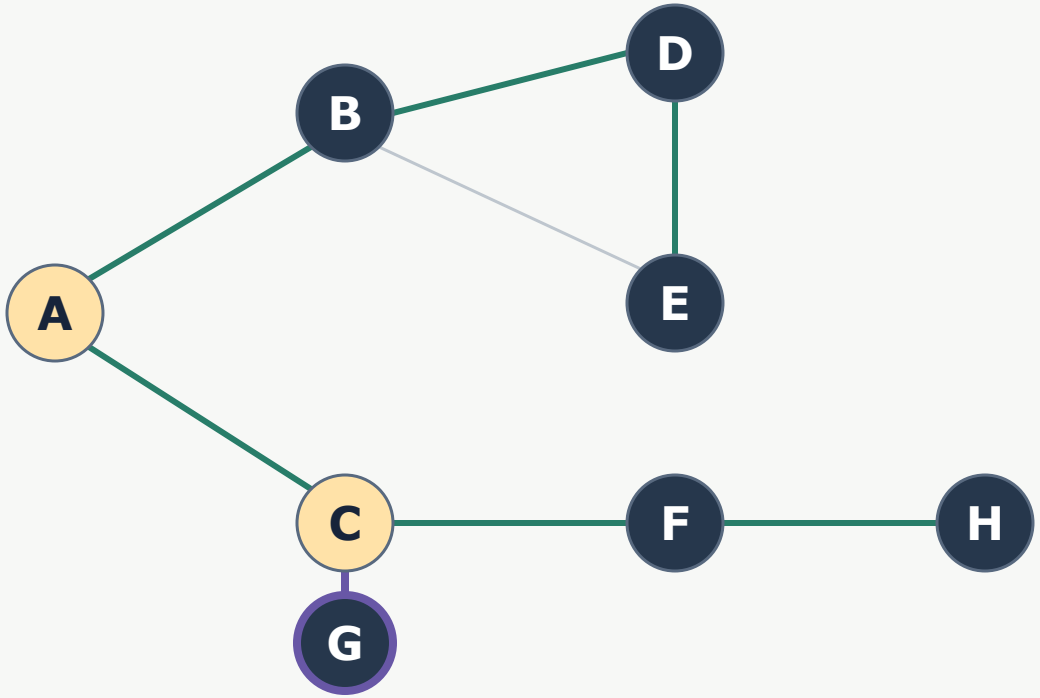
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	E	F	F	F	E	F
pai	—	A	A	B	D	C	C	F
d / f	1 / —	2 / 7	8 / —	3 / 6	4 / 5	9 / 12	13 / —	10 / 11

Descoberta: A, B, D, E, C, F, H, G

Término: E, D, B, H, F

15 Finalização de G

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(C) ← topo

visitar(A) base

Relógio: t = 14

G não possui vizinho novo.

G passa a finalizado e sai da pilha. C retoma a lista, agora no fim.

retorno da chamada

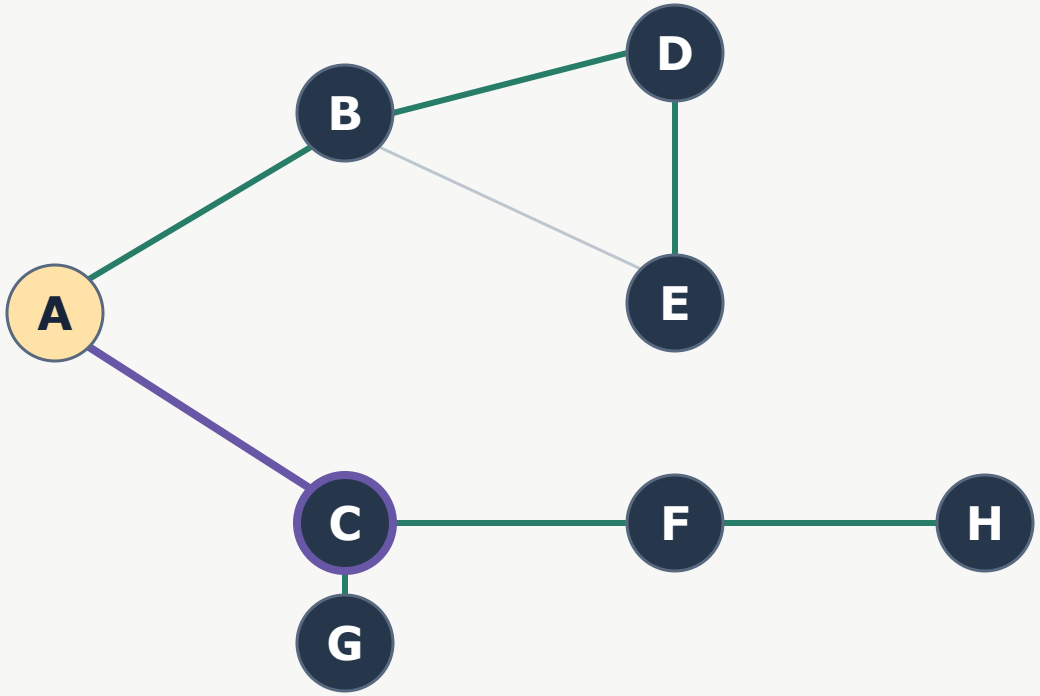
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	E	F	F	F	F	F
pai	—	A	A	B	D	C	C	F
d / f	1 / —	2 / 7	8 / —	3 / 6	4 / 5	9 / 12	13 / 14	10 / 11

Descoberta: A, B, D, E, C, F, H, G

Término: E, D, B, H, F, G

16 Finalização de C

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

visitar(A) ← topo base

Relógio: t = 15

Os ramos iniciados em F e G terminaram.

C passa a finalizado e sai da pilha. A retoma a lista, agora no fim.

retorno da chamada

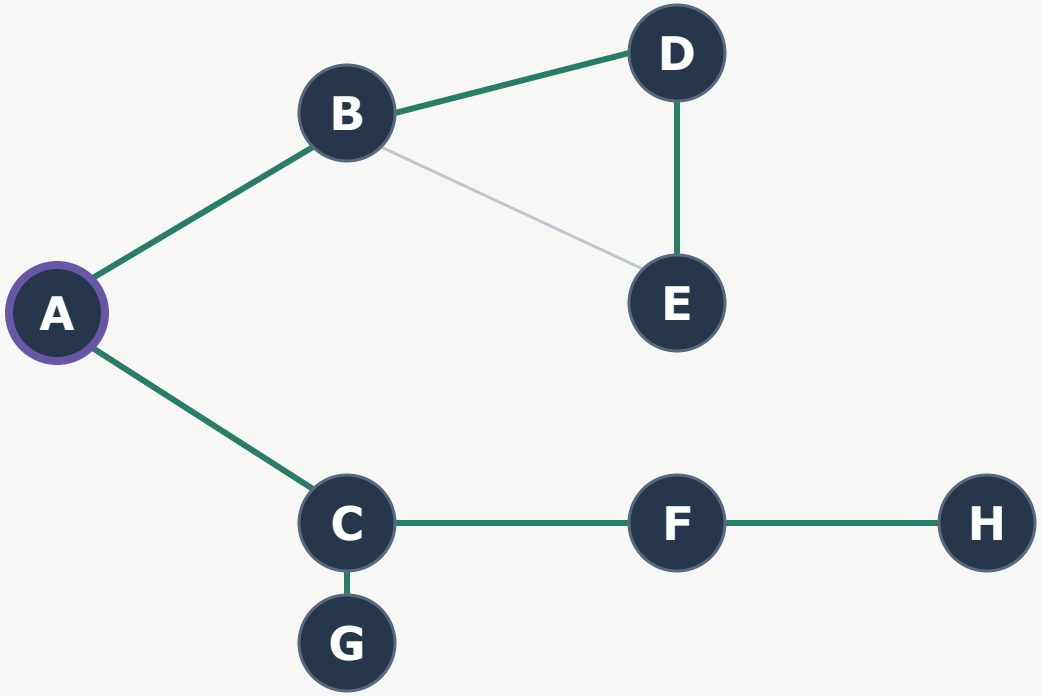
Vértice	A	B	C	D	E	F	G	H
Estado	E	F	F	F	F	F	F	F
pai	—	A	A	B	D	C	C	F
d / f	1 / —	2 / 7	8 / 15	3 / 6	4 / 5	9 / 12	13 / 14	10 / 11

Descoberta: A, B, D, E, C, F, H, G

Término: E, D, B, H, F, G, C

17 Finalização de A

N: não descoberto **E: em processamento** **F: finalizado** Verde: árvore; roxo: ação atual



Pilha de chamadas

Topo: chamada que executa agora

vazia

Relógio: t = 16

Os ramos de B e C terminaram. A também termina.

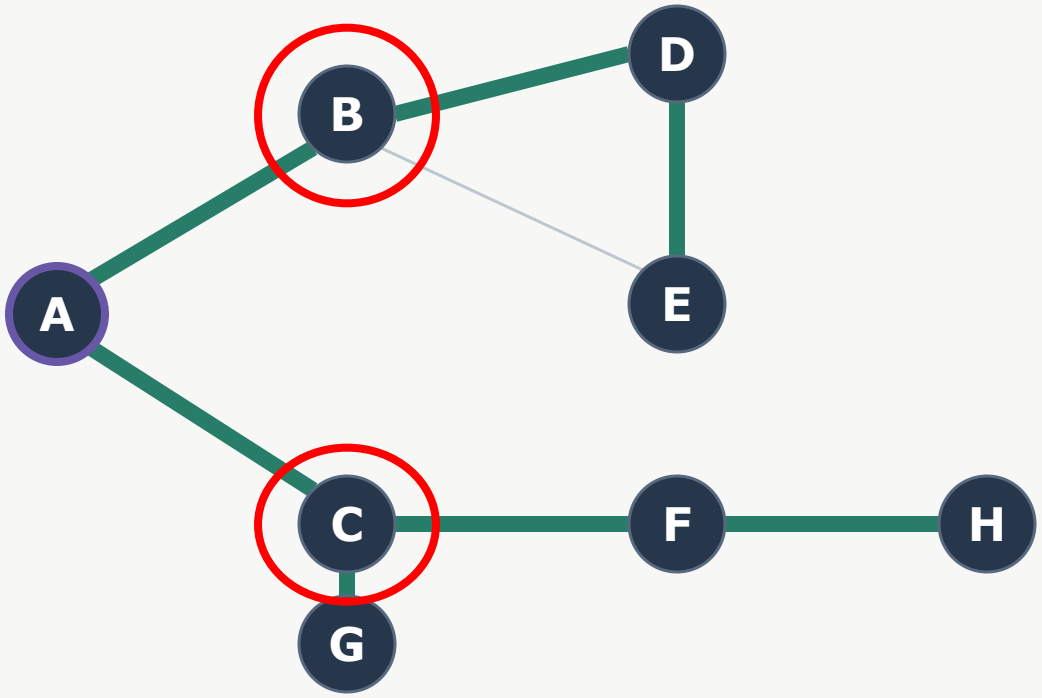
A pilha fica vazia. Como todos os vértices já foram descobertos, o percurso está completo.

Vértice	A	B	C	D	E	F	G	H
Estado	F	F	F	F	F	F	F	F
pai	—	A	A	B	D	C	C	F
d / f	1 / 16	2 / 7	8 / 15	3 / 6	4 / 5	9 / 12	13 / 14	10 / 11

Descoberta: A, B, D, E, C, F, H, G

Término: E, D, B, H, F, G, C, A

Resultado e leitura da execução



Ordens diferentes

Descoberta: A, B, D, E, C, F, H, G

Término: E, D, B, H, F, G, C, A

A pilha guarda o caminho ativo

Ao terminar B, a chamada de A continua em C. Ao terminar F, a chamada de C continua em G.

Arestas verdes formam a árvore DFS. B–E pertence ao grafo e fecha um ciclo, mas não entra na árvore.

Pergunta: antes de descobrir G, quais chamadas ainda estão ativas?

Resposta: visitar(A) e visitar(C). A descoberta de G acrescenta visitar(G) ao topo.

Um procedimento externo para as duas versões

DFS(G) inicializa e percorre o grafo inteiro

```
DFS(G):
    t ← 0
    para cada v em V:
        estado[v] ← não_descoberto
        pai[v] ← nulo
    para cada v em V, na ordem dada:
        se estado[v] = não_descoberto:
            visitar(v)
```

Uma implementação por execução

visitar(v) será implementada como visitar_rec(v) ou visitar_iter(v).

O laço externo inicia uma nova árvore quando encontra outro vértice não descoberto.

No exemplo anterior, a primeira raiz é A. Ao terminar seu percurso, todos os vértices já estão finalizados.

Versão recursiva: descoberta e término

```
visitar_rec(u):
    descobrir(u)
    para cada v em adj[u], na ordem dada:
        se estado[v] = não_descoberto:
            pai[v] ← u
            visitar_rec(v)
    finalizar(u)
```

Operações comuns às duas versões

```
descobrir(u):
    estado[u] ← em_processamento
    t ← t + 1; d[u] ← t

finalizar(u):
    estado[u] ← finalizado
    t ← t + 1; f[u] ← t
```

A chamada de u fica suspensa enquanto visitar_rec(v) explora o ramo de v. Ao voltar, u continua seu laço.

O retorno preserva o progresso do laço

Mesmo instante do slide 11: d[E] = 4

Topo até a base	Situação	Continuação
visitar_rec(E)	executa	examinar B, depois D
visitar_rec(D)	aguarda E	fim da lista
visitar_rec(B)	aguarda D	examinar E
visitar_rec(A)	aguarda B	examinar C

E examina B e D, ambos já descobertos. E termina em t = 5. D retoma e termina em t = 6.

B retoma no vizinho E, que já está finalizado. Só depois B termina, em t = 7. A então continua em C.

O quadro explícito guarda o próximo vizinho

Quadro (u, i): vértice u e índice i do próximo vizinho

Índices começam em zero. O topo aparece na primeira linha.

u	adj[u]	i	Próxima ação
E	[B, D]	0	examinar B
D	[B, E]	2	finalizar D, após E
B	[A, D, E]	2	examinar E, após D
A	[B, C]	1	examinar C, após B

Antes de descer para um filho, o quadro do pai avança i. Esse avanço registra onde o laço continuará.

Versão iterativa com pilha de quadros

```
visitar_iter(s):
    descobrir(s); empilhar(P, (s, 0))
    enquanto P não estiver vazia:
        (u, i) ← consultar o topo de P
        se i = quantidade de vizinhos de u:
            finalizar(u); desempilhar(P)
        senão:
            v ← adj[u][i]
            topo(P).i ← i + 1
            se estado[v] = não_descoberto:
                pai[v] ← u
                descobrir(v)
                empilhar(P, (v, 0))
```

A pilha P começa vazia em cada visita de uma raiz. Os estados e o relógio continuam os da execução completa.

Consultar o topo

Mantém o quadro na pilha para continuar o mesmo laço.

Empilhar

Corresponde a iniciar uma chamada recursiva.

Desempilhar

Corresponde ao retorno da chamada, após finalizar u.

A iteração reproduz o retorno de E para D

Ação	Pilha da base ao topo	Relógio
E examina B e D	(A,1), (B,2), (D,2), (E,2)	t = 4
Finalizar e retirar E	(A,1), (B,2), (D,2)	t = 5
Finalizar e retirar D	(A,1), (B,2)	t = 6

Por que E não empilha B nem D?

Os dois já estão em processamento. O índice de E avança mesmo sem criar uma chamada.

Por que D termina logo depois?

Seu quadro já guarda i = 2, o tamanho de [B, E]. Não há outro vizinho para examinar.

B termina e A continua no ramo de C

Ação	Pilha da base ao topo	Efeito
B examina E, já finalizado	(A,1), (B,3)	t continua 6
Finalizar e retirar B	(A,1)	f[B] = 7
A examina C e avança i	(A,2)	pai[C] = A
Descobrir e empilhar C	(A,2), (C,0)	d[C] = 8

A continua em C porque seu quadro guardou i = 1 durante toda a exploração do ramo de B.

A recursão e a pilha de quadros produzem a mesma descoberta, os mesmos pais e os mesmos tempos d/f.

Pilha apenas de vértices: uma variante simples

Antes da visita: P vazia e visitado[v] = falso para todo v.

```
empilhar(P, s)
enquanto P não estiver vazia:
    u ← desempilhar(P)
    se u ainda não foi visitado:
        marcar u como visitado; processar(u)
        para v em adj[u] na ordem inversa:
            se v ainda não foi visitado:
                empilhar(P, v)
```

O que essa variante registra

A descoberta de cada vértice quando ele sai da pilha pela primeira vez.

A pilha guarda candidatos pendentes, que podem se repetir.

No exemplo, antes de visitar E, a pilha pode conter [C, E, E], da base ao topo.

Essa pilha não representa o caminho de chamadas ativas. O algoritmo acima não registra os términos f[u].

Complexidade e memória das versões

Implementação	Tempo com listas	Memória auxiliar
Recursiva	$\Theta(\|V\| + \|E\|)$	$O(\|V\|)$
Iterativa com quadros	$\Theta(\|V\| + \|E\|)$	$O(\|V\|)$
Só vértices, com duplicatas	$\Theta(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$

Na recursão e na versão com quadros

Cada vértice entra e sai da pilha uma vez. Cada entrada das listas de adjacência é examinada uma vez. Em um grafo não dirigido, cada aresta aparece em duas listas.

Com matriz de adjacências: $\Theta(|V|^2)$. A memória indicada é adicional à representação do grafo.

Busca em largura e profundidade

Critério	Busca em largura	DFS recursiva ou com quadros
Estrutura	Fila FIFO	Pilha LIFO
Exploração	Camadas de distância	Ramos e retrocessos
Caminho mínimo sem pesos	Sim	Sem garantia
Informação de término	Não é o foco do percurso	Registro f[u]
Tempo com listas	$\Theta(\|V\| + \|E\|)$	$\Theta(\|V\| + \|E\|)$

As duas versões de DFS diferem em quem administra a pilha: a linguagem ou o próprio programa.

Conferência no mesmo grafo do exemplo

Vértice	A	B	C	D	E	F	G	H
pai	—	A	A	B	D	C	C	F
d	1	2	8	3	4	9	13	10
f	16	7	15	6	5	12	14	11

Descoberta: A, B, D, E, C, F, H, G

Término: E, D, B, H, F, G, C, A

Com a mesma ordem de raízes e vizinhos, a recursão e a pilha de quadros devem reproduzir toda a tabela.

Referências: Cormen et al., Introduction to Algorithms, 4ª ed., cap. 20.
Sedgewick e Wayne, Algorithms, 4ª ed., seção 4.1.